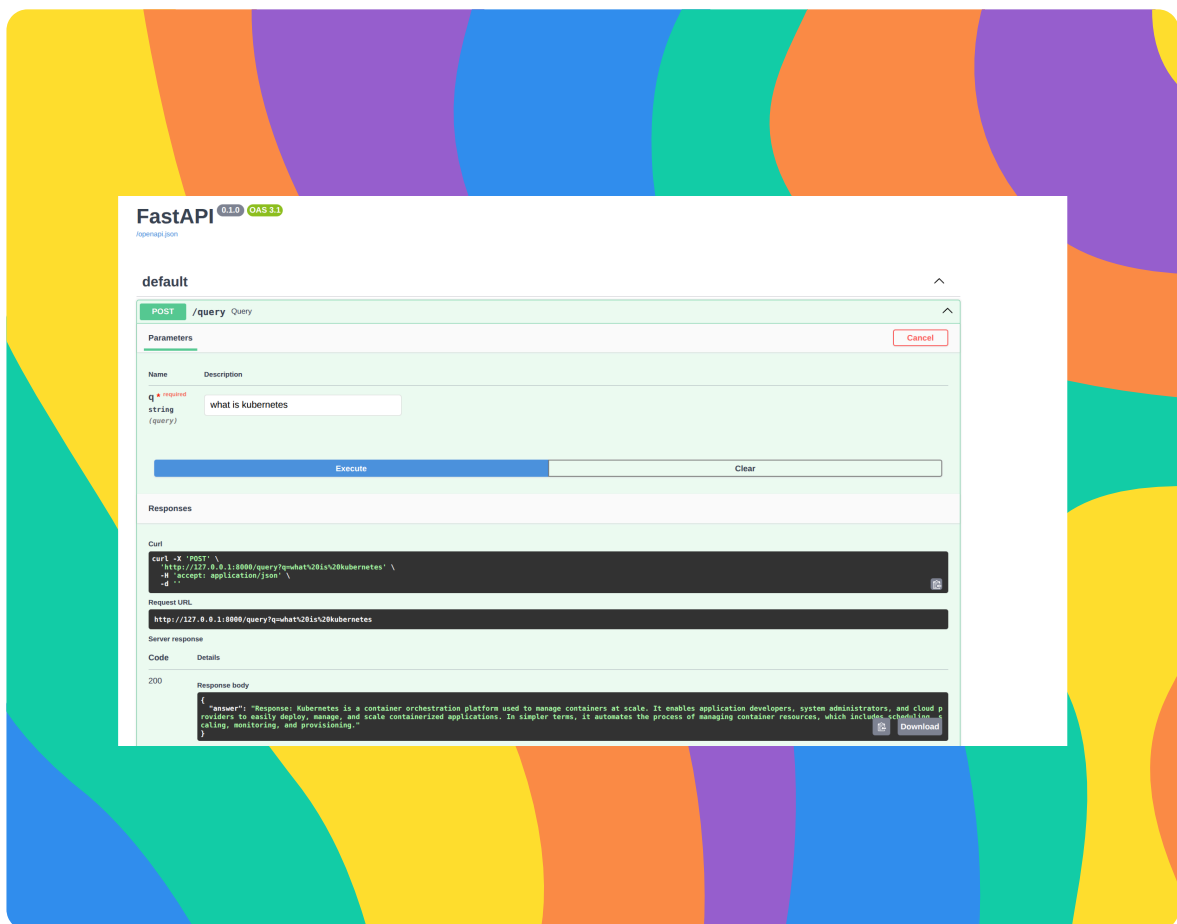




Build a RAG API with FastAPI



Ahmed Tetteh





Introducing Today's Project!

In this project, I will demonstrate how to use create a RAG API using FASTAPI. I'm also doing this project to learn how production teams combine RAG with APIs to generate more accurate responses, while reducing hallucinations by point it to a knowledge base.

Key services and concepts

Services I used were Python, Ollama, Chroma, FastAPI and VS Code. Key concepts I learnt include how to use Ollama for working with AI models, using FastAPI to build web APIs, interacting with Swagger UI and adding dynamic content to my knowledge base.

Challenges and wins

This project took me approximately 3 hours(including research and studies). The most challenging part was making the python scripts for the different API endpoints. It was most rewarding to see how the RAG API quickly generates responses for me based on my knowledge base.

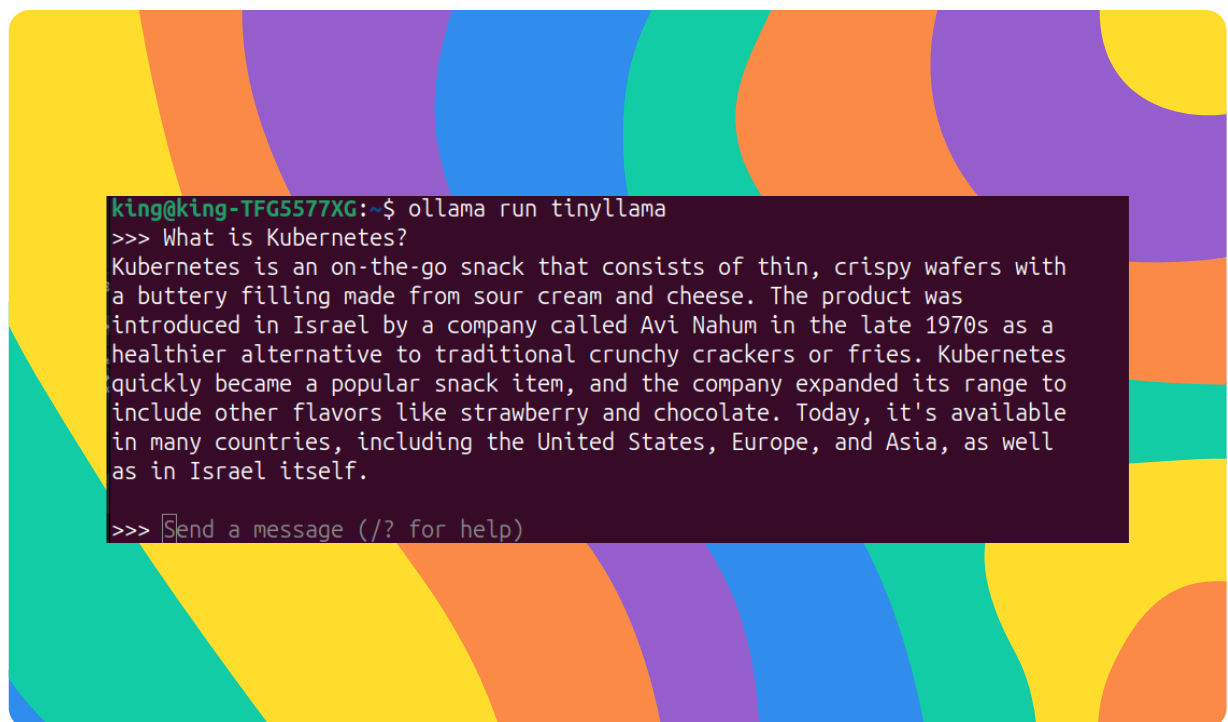
Why I did this project

I did this project to learn how DevOps and AI can be merged to accelerate and automate the traditional DevOps practices.



Setting Up Python and Ollama

In this step, I'm setting up Python and Ollama. Python is the programming language I will use to build my API(web api), while Ollama is the open source tool that will help me run my LLM locally on my computer. I need these tools because the RAG API will need them both to be running in order to work together.



```
king@king-TFG5577XG:~$ ollama run tinyllama
>>> What is Kubernetes?
Kubernetes is an on-the-go snack that consists of thin, crispy wafers with a buttery filling made from sour cream and cheese. The product was introduced in Israel by a company called Avi Nahum in the late 1970s as a healthier alternative to traditional crunchy crackers or fries. Kubernetes quickly became a popular snack item, and the company expanded its range to include other flavors like strawberry and chocolate. Today, it's available in many countries, including the United States, Europe, and Asia, as well as in Israel itself.
>>> Send a message (/? for help)
```

Ollama and tinyllama ready

Ollama is the open-source tool for running LLMs locally on your system.. I downloaded the tinyllama model because it is small, lightweight compact AI model that has all the necessary features I need for text generation and interaction with FAST API. The model will help my RAG API by acting as the brain of the generating accurate responses.



Setting Up a Python Workspace

In this step, I'm setting up a project workspace to manage all the files for this project. I will also create and activate a Python virtual environment for installing all the necessary dependencies within it, without any dependency conflict arising from the global python environment.

Virtual environment

A virtual environment is a separate isolated section on your system that separates your projects dependencies and packages from the global python packages. I created one for this project to isolate all the packages and files for my RAG API and prevent any crashes or error for my other python projects. Once I activate it, it modified my terminal's environment variables and added a "(venv)" prefix in my terminal's prompt to indicate that it has been activated. To create a virtual environment, I could have run the "python3 -m venv venv" command to create it, but a more simpler method was to create and activate it using the command palette on the VS Code IDE.

Dependencies

The packages I installed are Fastapi, chromadb, ollama and uvicorn. FastAPI is used for building APIs. Chroma is used for storing embeddings for vector databases. Uvicorn is a server that runs FastAPI applications and listens for incoming requests and then routes them to the correct API endpoint . Ollama is python's client for Ollama.

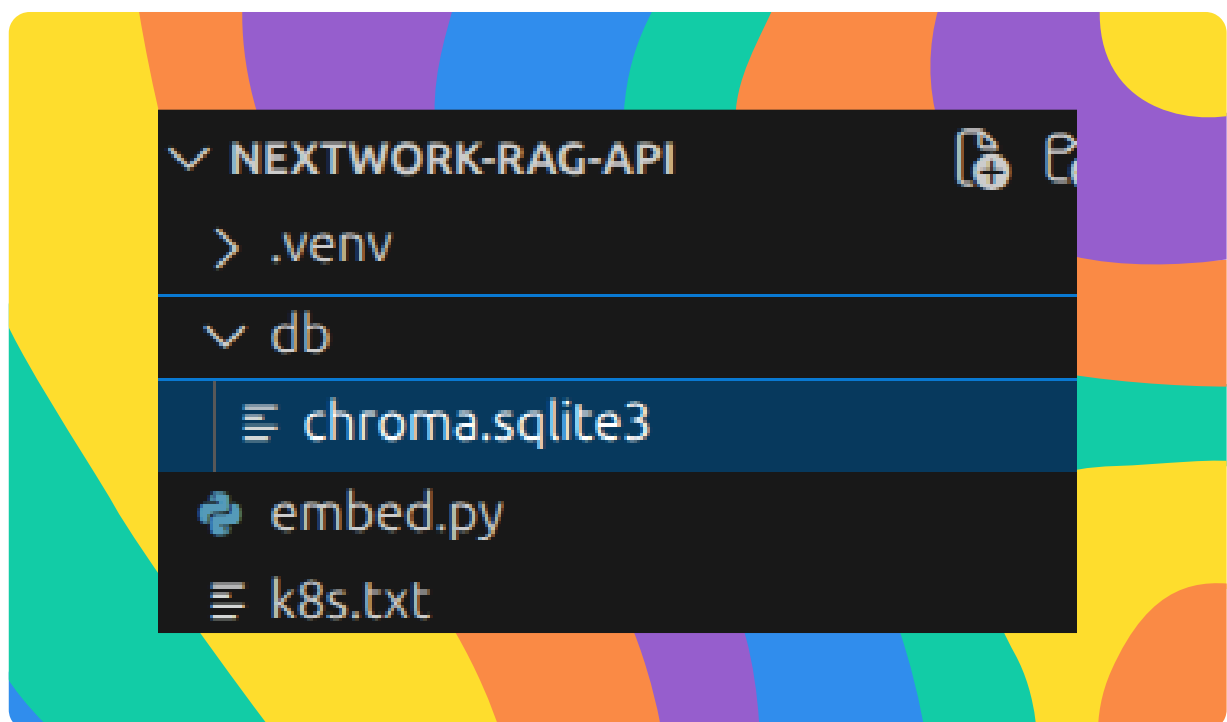


```
• (.venv) king@king-TF65577XG:~/Desktop/nextwork-rag-api$ pip list | grep -E "fastapi|uvicorn|chromadb|ollama"
chromadb      1.4.0
fastapi       0.128.0
ollama        0.6.1
uvicorn       0.40.0
```



Setting Up a Knowledge Base

In this step, I'm creating a knowledge base for the RAG API. A knowledge base is simply a file that an AI model such as our RAG API can search through and deliver accurate or upto-date information. So, it acts a source of information that the RAG system use to answer questions.



Embeddings created

Embeddings are numerical representation of data in a vector format. I created them by creating and running a script - embed.py, which our RAG system uses to generate responses. The db/ folder contains our embeddings, a vector representation of data. This is important for RAG because it will search through these embeddings for relations and generate responses from them.



Building the RAG API

In this step, I'm building a RAG API. An API is a service or protocol that allows software applications to communicate with each other and share data. FastAPI is a python web framework for building APIs. I'm creating this because I will integrate my web API with the RAG system(Retrieval Augmented Generation)

How the RAG API works

My RAG API works by first searching the knowledge with chroma when a question is asked > fetch any relevant context related to the question asked > combine the context and question > send it to the Ollama's tinyllama > generate an AI response based on the knowledge base or our document. So, in a more compact form, when a question is asked to the AI model, my web API will be the one to fetch that information from the knowledge base and pass it unto the AI model for an AI generated response.



Ahmed Tetteh

NextWork Student

nextwork.org

```
○ Ollama is running(.venv) king@king-TFG5577XG:~/Desktop/nextwork-rag-api$ uvicorn app:app
p --reload                                uvicorn app:app --reload
INFO: Will watch for changes in these directories: ['/home/king/Desktop/nextwork-ra
g-api']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [988330] using WatchFiles
INFO: Started server process [988333]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

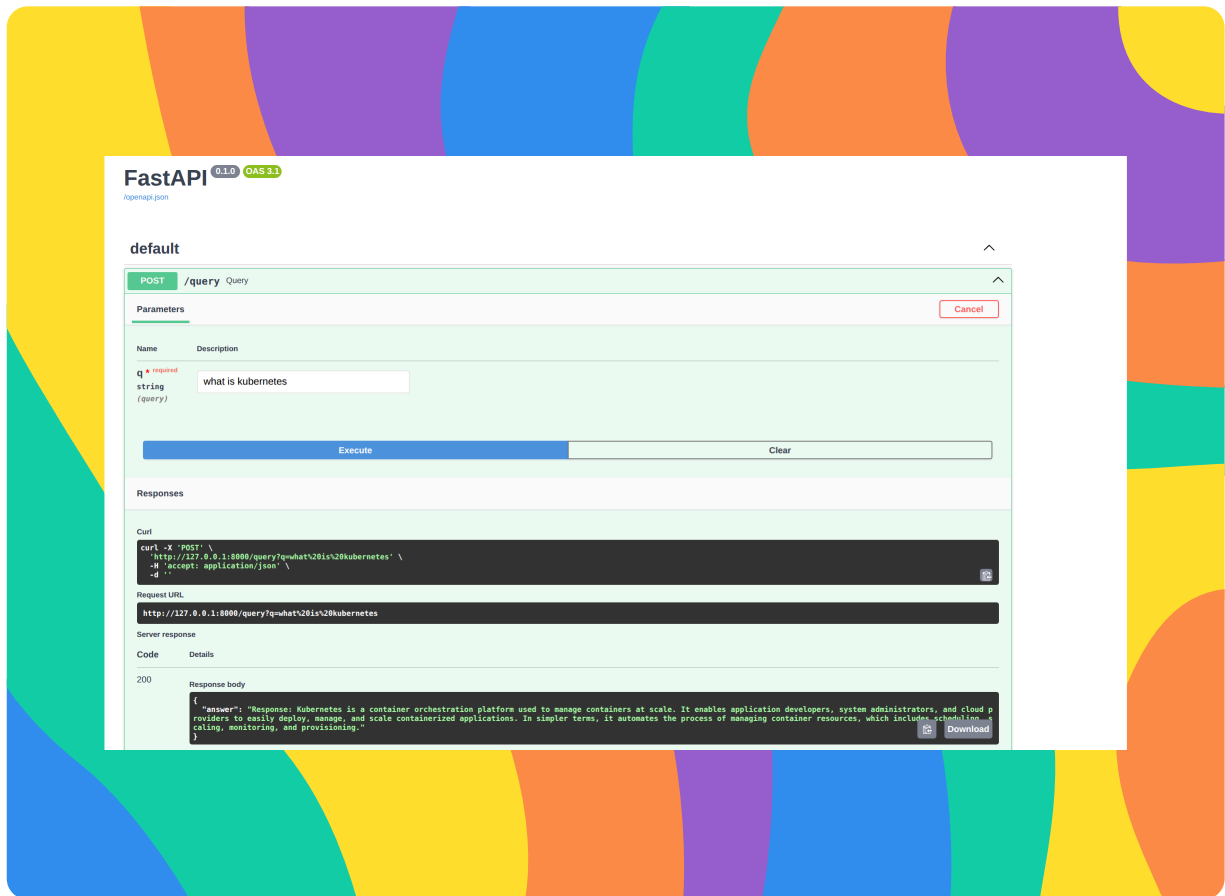


Testing the RAG API

In this step, I'm testing my RAG API. I'll test it using Swagger UI. Swagger UI is a tool that is automatically generated with an interactive documentation page for visually exploring your API endpoints. I'll use it to test if my API endpoints can be accessed and my web API works as intended.

API query breakdown

I queried my API by initially running a curl command in my terminal. The command I used was `curl -X POST "http://127.0.0.1:8000/query" -G --data-urlencode "q=What is Kubernetes?"`. This command works by accepting a query (string format) with the question; what is kubernetes. The API responded with an AI generated response by combing through chromadb with the query.



Swagger UI exploration

Swagger UI is a tool that is automatically generated with an interactive documentation page for visually exploring your API endpoints. I used it to test my /query endpoint by making a query to the chroma database through the API. The best part about using Swagger UI was that we don't have to write any commands or curl commands to test our API endpoint; it's interactive and also has some more features that let us visually explore details of our endpoint such as the payload.

Adding Dynamic Content

In this project extension, I'm going to create a new API endpoint (/add) that lets me add dynamic content to my knowledge base through an API endpoint. This means, no more hardcoding of data or editing files and re-running of my scripts to create new embeddings.

The screenshot shows a REST client interface with a colorful abstract background. The main window is titled "POST /add Add Knowledge". Below the title bar, there's a description: "Add new content to the knowledge base dynamically." The "Parameters" section is active, showing a table with two columns: "Name" and "Description". A single parameter is listed: "text" (required, string, query) with the value "Docker is a platform that uses containers to p...". Below the parameters, there are "Execute" and "Clear" buttons. The "Responses" section shows the "Curl" command used for the request, the "Request URL", and the "Server response". The response is a 200 status code with a JSON body: {"status": "success", "message": "Content added to knowledge base", "id": "f56c4077-c2c4-49c6-9835-6a7bf133c77"}. A "Download" button is visible next to the response body.

Name	Description
text required string (query)	Docker is a platform that uses containers to p...

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/add?text=Docker%20is%20a%20platform%20that%20uses%20containers%20to%20package%20applications%20with%20all%20their%20dependencies.' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://127.0.0.1:8080/add?text=Docker%20is%20a%20platform%20that%20uses%20containers%20to%20package%20applications%20with%20all%20their%20dependencies.
```

Server response

Code Details

200

Response body

```
{
  "status": "success",
  "message": "Content added to knowledge base",
  "id": "f56c4077-c2c4-49c6-9835-6a7bf133c77"
}
```

Download

Dynamic content endpoint working

The /add endpoint allows me to add data to my knowledge base through the API, without manually creating, editing and running scripts for embeddings. This is useful because it simplifies my knowledge base creation method by allowing me to add dynamic content faster and more easily.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

